

Each hash function h_{ab} maps $\mathbb{Z}_p$ to $\mathbb{Z}_m$. This family of hash functions has the nice property that the size m of the output range (which is the size of the hash table) is arbitrary—it need not be prime. Since you can choose from among $p - 1$ values for a and p values for b , the family $\mathcal{H}_{pm}$ contains $p(p - 1)$ hash functions.

Theorem 11.4

The family $\mathcal{H}_{pm}$ of hash functions defined by equations (11.3) and (11.4) is universal.

Proof Consider two distinct keys k_1 and k_2 from $\mathbb{Z}_p$, so that $k_1 \neq k_2$. For a given hash function h_{ab} , let

$$\begin{aligned} r_1 &= (ak_1 + b) \bmod p, \\ r_2 &= (ak_2 + b) \bmod p. \end{aligned}$$

We first note that $r_1 \neq r_2$. Why? Since we have $r_1 - r_2 = a(k_1 - k_2) \pmod{p}$, it follows that $r_1 \neq r_2$ because p is prime and both a and $(k_1 - k_2)$ are nonzero modulo p . By Theorem 31.6 on page 908, their product must also be nonzero modulo p . Therefore, when computing any $h_{ab} \in \mathcal{H}_{pm}$, distinct inputs k_1 and k_2 map to distinct values r_1 and r_2 modulo p , and there are no collisions yet at the “mod p level.” Moreover, each of the possible $p(p - 1)$ choices for the pair (a, b) with $a \neq 0$ yields a *different* resulting pair (r_1, r_2) with $r_1 \neq r_2$, since we can solve for a and b given r_1 and r_2 :

$$\begin{aligned} a &= ((r_1 - r_2)((k_1 - k_2)^{-1} \bmod p)) \bmod p, \\ b &= (r_1 - ak_1) \bmod p, \end{aligned}$$

where $((k_1 - k_2)^{-1} \bmod p)$ denotes the unique multiplicative inverse, modulo p , of $k_1 - k_2$. For each of the p possible values of r_1 , there are only $p - 1$ possible values of r_2 that do not equal r_1 , making only $p(p - 1)$ possible pairs (r_1, r_2) with $r_1 \neq r_2$. Therefore, there is a one-to-one correspondence between pairs (a, b) with $a \neq 0$ and pairs (r_1, r_2) with $r_1 \neq r_2$. Thus, for any given pair of distinct inputs k_1 and k_2 , if we pick (a, b) uniformly at random from $\mathbb{Z}_p^* \times \mathbb{Z}_p$, the resulting pair (r_1, r_2) is equally likely to be any pair of distinct values modulo p .

Therefore, the probability that distinct keys k_1 and k_2 collide is equal to the probability that $r_1 = r_2 \pmod{m}$ when r_1 and r_2 are randomly chosen as distinct values modulo p . For a given value of r_1 , of the $p - 1$ possible remaining values for r_2 , the number of values r_2 such that $r_2 \neq r_1$ and $r_2 = r_1 \pmod{m}$ is at most

$$\begin{aligned} \left\lceil \frac{p}{m} \right\rceil - 1 &\leq \frac{p + m - 1}{m} - 1 \quad (\text{by inequality (3.7) on page 64}) \\ &= \frac{p - 1}{m}. \end{aligned}$$

The probability that r_2 collides with r_1 when reduced modulo m is at most $((p-1)/m)/(p-1) = 1/m$, since r_2 is equally likely to be any of the $p-1$ values in $\mathbb{Z}_p$ that are different from r_1 , but at most $(p-1)/m$ of those values are equivalent to r_1 modulo m .

Therefore, for any pair of distinct values $k_1, k_2 \in \mathbb{Z}_p$,

$$\Pr \{h_{ab}(k_1) = h_{ab}(k_2)\} \leq 1/m ,$$

so that $\mathcal{H}_{pm}$ is indeed universal. ■

A $2/m$ -universal family of hash functions based on the multiply-shift method

We recommend that in practice you use the following hash-function family based on the multiply-shift method. It is exceptionally efficient and (although we omit the proof) provably $2/m$ -universal. Define $\mathcal{H}$ to be the family of multiply-shift hash functions with odd constants a :

$$\mathcal{H} = \{h_a : a \text{ is odd}, 1 \leq a < m, \text{ and } h_a \text{ is defined by equation (11.2)}\} . \quad (11.5)$$

Theorem 11.5

The family of hash functions $\mathcal{H}$ given by equation (11.5) is $2/m$ -universal. ■

That is, the probability that any two distinct keys collide is at most $2/m$. In many practical situations, the speed of computing the hash function more than compensates for the higher upper bound on the probability that two distinct keys collide when compared with a universal hash function.

11.3.5 Hashing long inputs such as vectors or strings

Sometimes hash function inputs are so long that they cannot be easily encoded modulo a reasonably sized prime number p or encoded within a single word of, say, 64 bits. As an example, consider the class of vectors, such as vectors of 8-bit bytes (which is how strings in many programming languages are stored). A vector might have an arbitrary nonnegative length, in which case the length of the input to the hash function may vary from input to input.

Number-theoretic approaches

One way to design good hash functions for variable-length inputs is to extend the ideas used in Section 11.3.4 to design universal hash functions. Exercise 11.3-6 explores one such approach.

Cryptographic hashing

Another way to design a good hash function for variable-length inputs is to use a hash function designed for cryptographic applications. *Cryptographic hash functions* are complex pseudorandom functions, designed for applications requiring properties beyond those needed here, but are robust, widely implemented, and usable as hash functions for hash tables.

A cryptographic hash function takes as input an arbitrary byte string and returns a fixed-length output. For example, the NIST standard deterministic cryptographic hash function SHA-256 [346] produces a 256-bit (32-byte) output for any input.

Some chip manufacturers include instructions in their CPU architectures to provide fast implementations of some cryptographic functions. Of particular interest are instructions that efficiently implement rounds of the Advanced Encryption Standard (AES), the “AES-NI” instructions. These instructions execute in a few tens of nanoseconds, which is generally fast enough for use with hash tables. A message authentication code such as CBC-MAC based on AES and the use of the AES-NI instructions could be a useful and efficient hash function. We don’t pursue the potential use of specialized instruction sets further here.

Cryptographic hash functions are useful because they provide a way of implementing an approximate version of a random oracle. As noted earlier, a random oracle is equivalent to an independent uniform hash function family. From a theoretical point of view, a random oracle is an unachievable ideal: a deterministic function that provides a randomly selected output for each input. Because it is deterministic, it provides the same output if queried again for the same input. From a practical point of view, constructions of hash function families based on cryptographic hash functions are sensible substitutes for random oracles.

There are many ways to use a cryptographic hash function as a hash function. For example, we could define

$$h(k) = \text{SHA-256}(k) \bmod m .$$

To define a family of such hash functions one may prepend a “salt” string a to the input before hashing it, as in

$$h_a(k) = \text{SHA-256}(a \parallel k) \bmod m ,$$

where $a \parallel k$ denotes the string formed by concatenating the strings a and k . The literature on message authentication codes (MACs) provides additional approaches.

Cryptographic approaches to hash-function design are becoming more practical as computers arrange their memories in hierarchies of differing capacities and speeds. Section 11.5 discusses one hash-function design based on the RC6 encryption method.

Exercises**11.3-1**

You wish to search a linked list of length n , where each element contains a key k along with a hash value $h(k)$. Each key is a long character string. How might you take advantage of the hash values when searching the list for an element with a given key?

11.3-2

You hash a string of r characters into m slots by treating it as a radix-128 number and then using the division method. You can represent the number m as a 32-bit computer word, but the string of r characters, treated as a radix-128 number, takes many words. How can you apply the division method to compute the hash value of the character string without using more than a constant number of words of storage outside the string itself?

11.3-3

Consider a version of the division method in which $h(k) = k \bmod m$, where $m = 2^p - 1$ and k is a character string interpreted in radix 2^p . Show that if string x can be converted to string y by permuting its characters, then x and y hash to the same value. Give an example of an application in which this property would be undesirable in a hash function.

11.3-4

Consider a hash table of size $m = 1000$ and a corresponding hash function $h(k) = \lfloor m(kA \bmod 1) \rfloor$ for $A = (\sqrt{5} - 1)/2$. Compute the locations to which the keys 61, 62, 63, 64, and 65 are mapped.

★ 11.3-5

Show that any ϵ -universal family $\mathcal{H}$ of hash functions from a finite set U to a finite set Q has $\epsilon \geq 1/|Q| - 1/|U|$.

★ 11.3-6

Let U be the set of d -tuples of values drawn from $\mathbb{Z}_p$, and let $Q = \mathbb{Z}_p$, where p is prime. Define the hash function $h_b : U \rightarrow Q$ for $b \in \mathbb{Z}_p$ on an input d -tuple $\langle a_0, a_1, \dots, a_{d-1} \rangle$ from U as

$$h_b(\langle a_0, a_1, \dots, a_{d-1} \rangle) = \left(\sum_{j=0}^{d-1} a_j b^j \right) \bmod p,$$

and let $\mathcal{H} = \{h_b : b \in \mathbb{Z}_p\}$. Argue that $\mathcal{H}$ is ϵ -universal for $\epsilon = (d-1)/p$. (Hint: See Exercise 31.4-4.)

11.4 Open addressing

This section describes open addressing, a method for collision resolution that, unlike chaining, does not make use of storage outside of the hash table itself. In *open addressing*, all elements occupy the hash table itself. That is, each table entry contains either an element of the dynamic set or NIL. No lists or elements are stored outside the table, unlike in chaining. Thus, in open addressing, the hash table can “fill up” so that no further insertions can be made. One consequence is that the load factor α can never exceed 1.

Collisions are handled as follows: when a new element is to be inserted into the table, it is placed in its “first-choice” location if possible. If that location is already occupied, the new element is placed in its “second-choice” location. The process continues until an empty slot is found in which to place the new element. Different elements have different preference orders for the locations.

To search for an element, systematically examine the preferred table slots for that element, in order of decreasing preference, until either you find the desired element or you find an empty slot and thus verify that the element is not in the table.

Of course, you could use chaining and store the linked lists inside the hash table, in the otherwise unused hash-table slots (see Exercise 11.2-4), but the advantage of open addressing is that it avoids pointers altogether. Instead of following pointers, you compute the sequence of slots to be examined. The memory freed by not storing pointers provides the hash table with a larger number of slots in the same amount of memory, potentially yielding fewer collisions and faster retrieval.

To perform insertion using open addressing, successively examine, or *probe*, the hash table until you find an empty slot in which to put the key. Instead of being fixed in the order $0, 1, \dots, m-1$ (which implies a $\Theta(n)$ search time), the sequence of positions probed depends upon the key being inserted. To determine which slots to probe, the hash function includes the probe number (starting from 0) as a second input. Thus, the hash function becomes

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\} .$$

Open addressing requires that for every key k , the *probe sequence* $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ be a permutation of $\{0, 1, \dots, m-1\}$, so that every hash-table position is eventually considered as a slot for a new key as the table fills up. The HASH-INSERT procedure on the following page assumes that the elements in the hash table T are keys with no satellite information: the key k is identical to the element containing key k . Each slot contains either a key or NIL (if the slot is empty). The HASH-INSERT procedure takes as input a hash table T and a key k

that is assumed to be not already present in the hash table. It either returns the slot number where it stores key k or flags an error because the hash table is already full.

```

HASH-INSERT( $T, k$ )
1   $i = 0$ 
2  repeat
3       $q = h(k, i)$ 
4      if  $T[q] == \text{NIL}$ 
5           $T[q] = k$ 
6          return  $q$ 
7      else  $i = i + 1$ 
8  until  $i == m$ 
9  error “hash table overflow”

HASH-SEARCH( $T, k$ )
1   $i = 0$ 
2  repeat
3       $q = h(k, i)$ 
4      if  $T[q] == k$ 
5          return  $q$ 
6       $i = i + 1$ 
7  until  $T[q] == \text{NIL}$  or  $i == m$ 
8  return NIL

```

The algorithm for searching for key k probes the same sequence of slots that the insertion algorithm examined when key k was inserted. Therefore, the search can terminate (unsuccessfully) when it finds an empty slot, since k would have been inserted there and not later in its probe sequence. The procedure HASH-SEARCH takes as input a hash table T and a key k , returning q if it finds that slot q contains key k , or NIL if key k is not present in table T .

Deletion from an open-address hash table is tricky. When you delete a key from slot q , it would be a mistake to mark that slot as empty by simply storing NIL in it. If you did, you might be unable to retrieve any key k for which slot q was probed and found occupied when k was inserted. One way to solve this problem is by marking the slot, storing in it the special value DELETED instead of NIL. The HASH-INSERT procedure then has to treat such a slot as empty so that it can insert a new key there. The HASH-SEARCH procedure passes over DELETED values while searching, since slots containing DELETED were filled when the key being searched for was inserted. Using the special value DELETED, however, means that search times no longer depend on the load factor α , and for this reason chaining is

frequently selected as a collision resolution technique when keys must be deleted. There is a simple special case of open addressing, linear probing, that avoids the need to mark slots with DELETED. Section 11.5.1 shows how to delete from a hash table when using linear probing.

In our analysis, we assume *independent uniform permutation hashing* (also confusingly known as *uniform hashing* in the literature): the probe sequence of each key is equally likely to be any of the $m!$ permutations of $\langle 0, 1, \dots, m-1 \rangle$. Independent uniform permutation hashing generalizes the notion of independent uniform hashing defined earlier to a hash function that produces not just a single slot number, but a whole probe sequence. True independent uniform permutation hashing is difficult to implement, however, and in practice suitable approximations (such as double hashing, defined below) are used.

We'll examine both double hashing and its special case, linear probing. These techniques guarantee that $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ is a permutation of $\langle 0, 1, \dots, m-1 \rangle$ for each key k . (Recall that the second parameter to the hash function h is the probe number.) Neither double hashing nor linear probing meets the assumption of independent uniform permutation hashing, however. Double hashing cannot generate more than m^2 different probe sequences (instead of the $m!$ that independent uniform permutation hashing requires). Nonetheless, double hashing has a large number of possible probe sequences and, as you might expect, seems to give good results. Linear probing is even more restricted, capable of generating only m different probe sequences.

Double hashing

Double hashing offers one of the best methods available for open addressing because the permutations produced have many of the characteristics of randomly chosen permutations. *Double hashing* uses a hash function of the form

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m ,$$

where both h_1 and h_2 are *auxiliary hash functions*. The initial probe goes to position $T[h_1(k)]$, and successive probe positions are offset from previous positions by the amount $h_2(k)$, modulo m . Thus, the probe sequence here depends in two ways upon the key k , since the initial probe position $h_1(k)$, the step size $h_2(k)$, or both, may vary. Figure 11.5 gives an example of insertion by double hashing.

In order for the entire hash table to be searched, the value $h_2(k)$ must be relatively prime to the hash-table size m . (See Exercise 11.4-5.) A convenient way to ensure this condition is to let m be an exact power of 2 and to design h_2 so that it always produces an odd number. Another way is to let m be prime and to design h_2 so that it always returns a positive integer less than m . For example, you

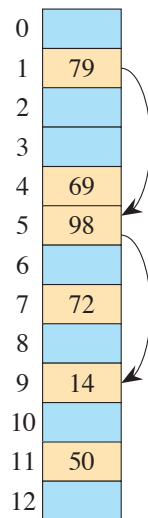


Figure 11.5 Insertion by double hashing. The hash table has size 13 with $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Since $14 = 1 \pmod{13}$ and $14 = 3 \pmod{11}$, the key 14 goes into empty slot 9, after slots 1 and 5 are examined and found to be occupied.

could choose m prime and let

$$h_1(k) = k \bmod m ,$$

$$h_2(k) = 1 + (k \bmod m') ,$$

where m' is chosen to be slightly less than m (say, $m - 1$). For example, if $k = 123456$, $m = 701$, and $m' = 700$, then $h_1(k) = 80$ and $h_2(k) = 257$, so that the first probe goes to position 80, and successive probes examine every 257th slot (modulo m) until the key has been found or every slot has been examined.

Although values of m other than primes or exact powers of 2 can in principle be used with double hashing, in practice it becomes more difficult to efficiently generate $h_2(k)$ (other than choosing $h_2(k) = 1$, which gives linear probing) in a way that ensures that it is relatively prime to m , in part because the relative density $\phi(m)/m$ of such numbers for general m may be small (see equation (31.25) on page 921).

When m is prime or an exact power of 2, double hashing produces $\Theta(m^2)$ probe sequences, since each possible $(h_1(k), h_2(k))$ pair yields a distinct probe sequence. As a result, for such values of m , double hashing appears to perform close to the “ideal” scheme of independent uniform permutation hashing.

Linear probing

Linear probing, a special case of double hashing, is the simplest open-addressing approach to resolving collisions. As with double hashing, an auxiliary hash function h_1 determines the first probe position $h_1(k)$ for inserting an element. If slot $T[h_1(k)]$ is already occupied, probe the next position $T[h_1(k) + 1]$. Keep going as necessary, on up to slot $T[m - 1]$, and then wrap around to slots $T[0]$, $T[1]$, and so on, but never going past slot $T[h_1(k) - 1]$. To view linear probing as a special case of double hashing, just set the double-hashing step function h_2 to be fixed at 1: $h_2(k) = 1$ for all k . That is, the hash function is

$$h(k, i) = (h_1(k) + i) \bmod m \quad (11.6)$$

for $i = 0, 1, \dots, m - 1$. The value of $h_1(k)$ determines the entire probe sequence, and so assuming that $h_1(k)$ can take on any value in $\{0, 1, \dots, m - 1\}$, linear probing allows only m distinct probe sequences.

We'll revisit linear probing in Section 11.5.1.

Analysis of open-address hashing

As in our analysis of chaining in Section 11.2, we analyze open addressing in terms of the load factor $\alpha = n/m$ of the hash table. With open addressing, at most one element occupies each slot, and thus $n \leq m$, which implies $\alpha \leq 1$. The analysis below requires α to be strictly less than 1, and so we assume that at least one slot is empty. Because deleting from an open-address hash table does not really free up a slot, we assume as well that no deletions occur.

For the hash function, we assume independent uniform permutation hashing. In this idealized scheme, the probe sequence $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$ used to insert or search for each key k is equally likely to be any permutation of $\langle 0, 1, \dots, m - 1 \rangle$. Of course, any given key has a unique fixed probe sequence associated with it. What we mean here is that, considering the probability distribution on the space of keys and the operation of the hash function on the keys, each possible probe sequence is equally likely.

We now analyze the expected number of probes for hashing with open addressing under the assumption of independent uniform permutation hashing, beginning with the expected number of probes made in an unsuccessful search (assuming, as stated above, that $\alpha < 1$).

The bound proven, of $1/(1 - \alpha) = 1 + \alpha + \alpha^2 + \alpha^3 + \dots$, has an intuitive interpretation. The first probe always occurs. With probability approximately α , the first probe finds an occupied slot, so that a second probe happens. With probability approximately α^2 , the first two slots are occupied so that a third probe ensues, and so on.

Theorem 11.6

Given an open-address hash table with load factor $\alpha = n/m < 1$, the expected number of probes in an unsuccessful search is at most $1/(1 - \alpha)$, assuming independent uniform permutation hashing and no deletions.

Proof In an unsuccessful search, every probe but the last accesses an occupied slot that does not contain the desired key, and the last slot probed is empty. Let the random variable X denote the number of probes made in an unsuccessful search, and define the event A_i , for $i = 1, 2, \dots$, as the event that an i th probe occurs and it is to an occupied slot. Then the event $\{X \geq i\}$ is the intersection of events $A_1 \cap A_2 \cap \dots \cap A_{i-1}$. We bound $\Pr\{X \geq i\}$ by bounding $\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\}$. By Exercise C.2-5 on page 1190,

$$\begin{aligned} \Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\} &= \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \\ &\quad \Pr\{A_{i-1} \mid A_1 \cap A_2 \cap \dots \cap A_{i-2}\}. \end{aligned}$$

Since there are n elements and m slots, $\Pr\{A_1\} = n/m$. For $j > 1$, the probability that there is a j th probe and it is to an occupied slot, given that the first $j - 1$ probes were to occupied slots, is $(n - j + 1)/(m - j + 1)$. This probability follows because the j th probe would be finding one of the remaining $(n - (j - 1))$ elements in one of the $(m - (j - 1))$ unexamined slots, and by the assumption of independent uniform permutation hashing, the probability is the ratio of these quantities. Since $n < m$ implies that $(n - j)/(m - j) \leq n/m$ for all j in the range $0 \leq j < m$, it follows that for all i in the range $1 \leq i \leq m$, we have

$$\begin{aligned} \Pr\{X \geq i\} &= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdots \frac{n-i+2}{m-i+2} \\ &\leq \left(\frac{n}{m}\right)^{i-1} \\ &= \alpha^{i-1}. \end{aligned}$$

The product in the first line has $i - 1$ factors. When $i = 1$, the product is 1, the identity for multiplication, and we get $\Pr\{X \geq 1\} = 1$, which makes sense, since there must always be at least 1 probe. If each of the first n probes is to an occupied slot, then all occupied slots have been probed. Then, the $(n + 1)$ st probe must be to an empty slot, which gives $\Pr\{X \geq i\} = 0$ for $i > n + 1$. Now, we use equation (C.28) on page 1193 to bound the expected number of probes:

$$\begin{aligned} E[X] &= \sum_{i=1}^{\infty} \Pr\{X \geq i\} \\ &= \sum_{i=1}^{n+1} \Pr\{X \geq i\} + \sum_{i>n+1} \Pr\{X \geq i\} \end{aligned}$$